NUS National University of Singapore

BT5110
Data Management and Warehousing

Schema Refinement: Functional Dependencies

Database Schema = Relation Schemas + Constraints

id	name	dept	position	salary
1XU3	Dewi Srijaya	Sales	Clerk	2000
SCT4	Axel Bayer	Marketing	Trainee	1200
4XR2	John Smith	Accounting	Clerk	2000
7HG5	Eric Wei	Sales	Assistant Manager	2200
4DE3	Winne Lee	Accounting	Manager	3000
8H5G	Sylvia Tan	Marketing	Manager	2900
Null	Null	Null	Security Guard	1500

- Table records the salaries of different employees, which are determined by the position.
- This business rule can be translated into an integrity constraint called functional dependency
  - {position} → {salary}

Good and Bad Schema Design

id	name	dept	position	salary
1XU3	Dewi Srijaya	Sales	Clerk	2000
SCT4	Axel Bayer	Marketing	Trainee	1200
4XR2	John Smith	Accounting	Clerk	2000
7HG5	Eric Wei	Sales	Assistant Manager	2200
4DE3	Winne Lee	Accounting	Manager	3000
8H5G	Sylvia Tan	Marketing	Manager	2900
Null	Null	Null	Security Guard	1500

- **With this design, information is repeated**

Good and Bad Schema Design

id	name	dept	position	salary
1XU3	Dewi Srijaya	Sales	Clerk	2000
SCT4	Axel Bayer	Marketing	Trainee	1200
4XR2	John Smith	Accounting	Clerk	2000
7HG5	Eric Wei	Sales	Assistant Manager	2200
4DE3	Winne Lee	Accounting	Manager	3000
8H5G	Sylvia Tan	Marketing	Manager	2900
Null	Null	Null	Security Guard	1500

- **With this design, the functional dependency {position} → {salary} can be easily violated**
- **Two managers have different salaries**

Good and Bad Schema Design

NUS National University of Singapore

id	name	dept	position	salary
1XU3	Dewi Srijaya	Sales	Clerk	2000
SCT4	Axel Bayer	Marketing	Trainee	1200
4XR2	John Smith	Accounting	Clerk	2000
7HG5	Eric Wei	Sales	Assistant Manager	2200
4DE3	Winne Lee	Accounting	Manager	3000
8H5G	Sylvia Tan	Marketing	Manager	2900
Null	Null	Null	Security Guard	1500

- **With this design, we cannot remember the salary of a security guard if we do not have at least one in the company unless we use null values**
- **This may not even be possible if the ID is a primary key in the table**

Good and Bad Schema Design

id	name	dept	position	salary
1XU3	Dewi Srijaya	Sales	Clerk	2000
SCT4	Axel Bayer	Marketing	Trainee	1200
4XR2	John Smith	Accounting	Clerk	2000
7HG5	Eric Wei	Sales	Assistant Manager	2200
4DE3	Winne Lee	Accounting	Manager	3000
8H5G	Sylvia Tan	Marketing	Manager	2900
Null	Null	Null	Security Guard	1500

- **With this design, we cannot accept Eric Wei's resignation – we will forget the salary of an assistant manager if Eric leaves.**

Good and Bad Schema Design

- A common solution is to decompose the relation
- Decomposition is lossless if the join of the two relations reconstructs back the original relation

Employee

id	name	dept	position
1XU3	Dewi Srijaya	Sales	Clerk
SCT4	Axel Bayer	Marketing	Trainee
4XR2	John Smith	Accounting	Clerk
7HG5	Eric Wei	Sales	Assistant Manager
4DE3	Winne Lee	Accounting	Manager
8H5G	Sylvia Tan	Marketing	Manager

Salary

position	salary
Clerk	2000
Trainee	1200
Assistant Manager	2200
Manager	2900
Security Guard	1500

Good and Bad Schema Design

id	name	dept	position	salary
1XU3	Dewi Srijaya	Sales	Clerk	2000
SCT4	Axel Bayer	Marketing	Trainee	1200
4XR2	John Smith	Accounting	Clerk	2000
7HG5	Eric Wei	Sales	Assistant Manager	2200
4DE3	Winne Lee	Accounting	Manager	3000
8H5G	Sylvia Tan	Marketing	Manager	2900

- **There are many other FDs in this application**
  - **{id} → {name}**
  - **{id} → {dept}**
  - **{id} → {position}**

Schema Design Issues

- **How to determine whether a schema design is good?**
- **How to transform a bad design into a good one?**
- **Theory of functional dependencies provide a systematic approach to address these issues**
  - Introduced by E.F. Codd
  - *A relational model for large shared data banks*, in Communications of the ACM, Vol. 13, No. 6, 1970.

Functional Dependencies (FDs)

- Let X and Y be subsets of attributes of a relation R.
- A functional dependency $X \rightarrow Y$ holds over relation R if and only if for any instance *r* of R, whenever two tuples t_1 and t_2 of *r* agree on the attributes X, they also agree on the attributes Y
  - $t_1.X = t_2.X \Rightarrow t_1.Y = t_2.Y$
- We say that X functionally determines Y
  (or Y functionally depends on X)

Example

- **MovieList (title, director, address, phone, time)**

title	director	address	phone	time
Schlinder's List	Spielberg	Holland	3355	1130
Saving Private Ryan	Spielberg	Holland	3355	1430
Noth by Northwest	Hitchcock	Orchard	1234	1400
The Godfather	Coppola	Orchard	1234	1700
Saving Private Ryan	Spielberg	Orchard	1234	2130

- **Functional dependencies on MovieList:**

  {title} $\rightarrow$ {director}

  {address} $\rightarrow$ {phone}

  {address, time} $\rightarrow$ {title}

FDs Definitions

- Let *r* be a relation instance of relation schema *R*
- *r* satisfies FD $X \rightarrow Y$ if for every pair of tuples t_1 and t_2 in *r* such that $t_1.X = t_2.X$, it is also true that $t_1.Y = t_2.Y$
- An FD *f* holds on *R* if and only if for any relation instance *r* of *R*, *r* satisfies *f*
- *r* is said to violate an FD *f* if *r* does not satisfy *f*
- An FD $X \rightarrow Y$ is a **trivial FD** if $Y \subseteq X$; otherwise it is a non-trivial FD

Example

NUS National University of Singapore

- **Given relation schema Movie (title, director, producer)**

title	director	producer
Saving Private Ryan	Spielberg	Williams
Noth by Northwest	Hitchcock	Harris
Schindler's List	Spielberg	Williams
Vertigo	Hitchcock	Harris

- **A relation instance *r* of Movie, satisfies the FD**
  {producer} → {director}
- **But we cannot conclude** {producer} → {director} **holds on Movie**
- **Based on instances of a relation R, we can tell which FDs do not hold on R, but we cannot deduce which non-trivial FDs hold on R!**

Quiz

Consider the relation instance r of schema R (A, B, C)

A	B	C
0	0	0
2	1	2
1	1	2
0	0	1

List all non-trivial FDs that are satisfied by r

A→B

C→B

Reasoning about FDs

- **Implication problem:**
  - Given a set of FDs *G* that hold on *R*, and an FD *f*, does *f* also hold on *R*?
- **Example in MovieList, we have the set of FDs**

  G = { {title $\rightarrow$ director}, {address $\rightarrow$ phone,}
  {address, time} $\rightarrow$ {title} }
- ***G* logically implies *f* if every relation instance *r* of *R* that satisfies the FDs in *G* also satisfies the FD *f***

Reasoning about FDs

- Let *G* and *H* denote sets of FDs, and *f* denote an FD
- *G* implies *H* if *G* implies *f* for each $f \in H$
- Closure of *G*, denoted by G^+, is the set of all FDs implied by *G*
- Two sets of FDs, *G* and *H*, are equivalent, denoted by $G \equiv H$, if $G^+ = H^+$

Axioms for FDs

- **A collection of formal rules used to derive an FD from a set of FDs**
- **Let *X, Y, Z* denote sets of attributes over a relation schema R**
- **Armstrong's Axioms**
  - **Reflexivity: If $Y \subseteq X$, then $X \rightarrow Y$**
  - **Augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$**
  - **Transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$**
- **Armstrong's Axioms are both sound and complete**
  - Sound: Any derived FD is implied by G
  - Complete: All FDs in G^+ can be derived

Example

Consider R(A, B, C, D, E) with FDs

$$G = \{ \{A \rightarrow C\}, \{B \rightarrow C\}, \{CD \rightarrow E\} \}$$

Show that G implies $\{AD\} \rightarrow \{E\}$

Example

Consider R(A, B, C, D, E) with FDs

G = { {A → C}, {B → C}, {CD → E} }

Show that G implies {AD} → {E}

1)	{A} → {C}	*Given*
2)	{AD} → {CD}	*Augmentation with (1)*
3)	{CD} → {E}	*Given*
4)	{AD} → {E}	*Transitivity with (2) and (3)*

Additional Inference Rules

- **Union:**

  If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$

- **Decomposition:**

  If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

Keys and Superkeys

- A set of attributes X is a **superkey** of schema R (with FDs G) if G implies $X \rightarrow R$
- A set of attributes X is a **key** of schema R if
  - X is a superkey, and
  - No proper subset of X is a superkey
- An attribute A in R is a **prime attribute** if A is contained in some key of R; otherwise, it is a **nonprime attribute**

Example

Consider relation schema

MovieList (title, director, address, phone, time)

with FDs { {address, time} → {title},

{address} → {phone}, {title} → {director} }

- {address, time} is the only key of MovieList
- {address, time} are the only prime attributes in MovieList
- Any superset of {address, time} is a superkey of MovieList

Example

Consider R(A, B, C, D) with FDs G = { {A} → {C}, {B} → {D} }

Is {AB} a superkey?

Example

Consider R(A, B, C, D) with FDs G = { {A} → {C}, {B} → {D} }

Is {AB} a superkey?

1) {AB} → {BC} Augmentation of {A} → {C} with {B}
2) {AB} → {ABC} Reflexivity and union with {A} → {A}
3) {ABC} → {ABCD} Augmentation of {B} → {D} with {ABC}
4) {AB} → {ABCD} Transitivity with (2) and (3)

Hence, {AB} is a superkey

Closure of a Set of FDs

- Computing G^+ for a set of FDs G is not efficient as the size of G^+ could be **exponentially large**
- Consider the relation scheme R(A,B,C,D)

  G = { {A} → {B}, {B,C} → {D} }

  G+ = { {A} → {A}, {B} → {B}, {C} → {C}, {D}→{D},
  {A} → {B}, {A,B} → {B}, {A,D}→ {B,D},
  {A,C} → {B,C}, {A,C,D} → {B,C,D},
  {A} → {A,B}, {A,B} → {A,B}, {A,D} → {A,B,D},
  {A,C} → {A,B,C}, {A,C,D} → {A,B,C,D},
  {B,C} → {D}, …, {A,C} → {D}, … }

Attribute Closure

- More efficient to compute the closure of a set of attributes
- Let $X \subseteq R$ and G be a set of FDs that hold on R
- Closure of X (with respect to G), denoted by X^+, is the set of attributes that are functionally determined by X with respect to G

Computing Attribute Closure

NUS National University of Singapore

Input: X, G

Output: X^+ w.r.t. G

Let $X_0 = X$ and $i = 0$

Repeat

$X_{i+1} = X_i \cup Z$ such that there is some FD $\{Y\} \rightarrow \{Z\} \in G$ and $Y \subseteq X_i$

Until when $X_{i+1} = X_i$

Return X_i

Example

Let G = { {AB} → {C}, {C} → {A}, {BC} → {D},
{ACD} → {B}, {D} → {EJ}, {BE} → {C},
{CJ} → {BD}, {CE} → {AJ} }

Compute the closure of {BD}

Example

NUS National University of Singapore

Let G = { {AB} → {C}, {C} → {A}, {BC} → {D},
{ACD} → {B}, {D} → {EJ}, {BE} → {C},
{CJ} → {BD}, {CE} → {AJ} }

Compute the closure of {BD}

i	X_i	FD used
0	{BD}	Given
1	{BDEJ}	{D} → {EJ}
2	{BCDEJ}	{BE} → {C}
3	{ABCDEJ}	{CE} → {AJ}
4	{ABCDEJ}	

Thus, $\{BD\}^+ = \{ABCDEJ\}$

Example

Let G = { {A} → {C}, {B} → {C}, {CD} → {E} }

Show that G implies {AD} → {E}

Example

Let G = { {A} → {C}, {B} → {C}, {CD} → {E} }

Show that G implies {AD} → {E}

i	X_i	FD used
0	AD	given
1	ACD	A → C
2	ACDE	CD → E
3	ACDE	none

Thus, $\{AD\}^+ = \{ACDE\}$

Since $E \in \{AD\}^+$, therefore G implies {AD} → {E}

Equivalence of Sets of FDs

- We can use attribute closure to determine if two sets of FDs G and H are equivalent
- For each FD $X \rightarrow Y \in G$
  - Compute X+ with respect to H
  - $X \rightarrow Y \in H^+$ if $Y \subseteq X+$
- Do the same for each FD in H

Redundant Attributes in FDs

- An attribute $A \in X$ is **redundant** in the FD $X \rightarrow B$ if $(G - \{X \rightarrow B\} \cup \{X - A\} \rightarrow B\})$ is equivalent to G
- How to check if $A \in X$ is redundant in $X \rightarrow B$?
  - Compute $(X - A)^+$ w.r.t. G
  - $A \in X$ is redundant in $X \rightarrow B$ if $B \in (X - A)^+$

Redundant Attributes in FDs

- **Find the redundant attributes in**

  { {AB} → {C}, {A} → {B}, {B} → {A} }

Redundant Attributes in FDs

- **Find the redundant attributes in**

  { {AB} → {C}, {A} → {B}, {B} → {A} }

  Check if B is redundant by computing attribute closure {A}+

  {A}+ = {B} ({A} → {B})

  {A}+ = {AB} ({B} → {A})

  {A}+ = {ABC} ({AB} → {C})

  B is redundant in {AB} → {C} since C is in {A}+

Redundant FDs

- FD $f \in G$ is redundant if $(G - \{ f \})$ is equivalent to G
- How to check if an FD $X \rightarrow A$ is redundant in G?
  - Compute X^+ w.r.t. $G - \{ X \rightarrow A \}$
  - $X \rightarrow A$ is redundant in F if $A \in X^+$

Redundant FDs

- Find the redundant FDs in G = { {A} → {B},
  {A} → {C}, {B} → {A}, {B} → {C}, {C} → {A} }

Redundant FDs

- **Find the redundant FDs in G = { {A} → {B}, {A} → {C}, {B} → {A}, {B} → {C}, {C} → {A} }**

Check if {A} → {B} is redundant

Compute A+ w.r.t. to G – { {A} → {B} }

A+ = {AC} ({A} → {C})

B is not in A+, so {A} → {B} is not redundant

Redundant FDs

- **Find the redundant FDs in G = { {A} → {B}, {A} → {C}, {B} → {A}, {B} → {C}, {C} → {A} }**

Check if {A} → {C} is redundant

Compute A+ w.r.t. to G – { {A} → {C} }

A+ = {AB} ({A} → {B})

A+ = {ABC} ({B} → {C})

C is in A+, so {A} → {C} is redundant

- **Update G = { {A} → {B}, ~~{A} → {C}~~, {B} → {A}, {B} → {C}, {C} → {A} }**

Redundant FDs

- **G = { {A} → {B}, {B} → {A}, {B} → {C}, {C} → {A} }**

Check if {B} → {A} is redundant

Compute B+ w.r.t. to G – { {B} → {A} }

B+ = {BC} ({B} → {C})

B+ = {ABC} ({C} → {A})

A is in B+, so {B} → {A} is redundant

- **Update G =**

{ {A} → {B}, ~~{B} → {A}~~, {B} → {C}, {C} → {A} }

Redundant FDs

- **G = { {A} → {B}, {B} → {C}, {C} → {A} }**

Check if {B} → {C} is redundant

Compute B+ w.r.t. to G – { {B} → {C} }

B+ = {B}

C is not in B+, so {B} → {C} is not redundant

Redundant FDs

- **G = { {A} → {B}, {B} → {C}, {C} → {A} }**

Finally, check if {C} → {A} is redundant

Compute C+ w.r.t. to G – { {C} → {A} }

C+ = {C}

A is not in C+, so {C} → {A} is not redundant

Minimal Cover for FDs

- A set of FDs G is a minimal cover for a set of FDs H if and only if
  - Every FD in G is of the form $X \rightarrow A$ where X is a set of attributes, A is a single attribute, and X has no redundant attributes
  - There are no redundant FDs in G
  - G and H are equivalent

Computing Minimal Cover

- **Algorithm**
  - Use decomposition rule to obtain FDs with one attribute on RHS
  - Remove redundant attributes from LHS of each FD
  - Remove redundant FDs
- **Minimal covers may not be unique due to choice of redundant attributes/FDs**

Example

NUS National University of Singapore

Let G = { {ABCD} → {E}, {E} → {D}, {A} → {B}, {AC} → {D} }

Find a minimal cover of G.

Example

Let G = { {ABCD} → {E}, {E} → {D}, {A} → {B}, {AC} → {D} }

Find a minimal cover of G.

1. **Decompose FDs**
   - All FDs in G have a single attribute on the RHS
2. **Eliminate redundant attributes**
   - B in {ABCD} → {E} is redundant since E ∈ {ACD}$^+$ wrt G
   - G = { {ACD} → {E}, {E} → {D}, {A} → {B}, {AC} → {D} }
   - D in {ACD} → {E} is redundant since E ∈ {AC}$^+$ wrt G
   - G = { {AC} → {E}, {E} → {D}, {A} → {B}, {AC} → {D} }
   - There are no more redundant attributes in G

Example

G = { {AC} → {E}, {E} → {D}, {A} → {B}, {AC} → {D} }

3. Eliminate redundant FDs

- {AC} → {D} is redundant
  since D ∈ {AC}$^+$ wrt G – {AC → D}
- G = {AC → E, E → D, A → B}
- There are no more redundant FDs in G

A minimal cover of G is

{ {AC} → {E}, {E} → {D}, {A} → {B} }

Summary

- Good and bad schema design
- Functional dependencies in relation schema
- Reasoning and axioms for FDs
- Attribute closure
- Algorithm to compute minimal cover